

Clustering

Clustering is an unsupervised machine learning technique used to group similar data points together into clusters. why it comes under Unsupervised learning mean since it does not contain any target/output column

“Clustering is the process of dividing data into groups based on similarity.”

Commonly used in:

- Customer segmentation (grouping customers by behaviours)
- Image recognition
- Market research

Clustering Methods:

Common clustering methods are K-Means, Hierarchical Clustering(Agglomerative &, DBSCAN, Mean Shift, Gaussian Mixture Model, and Fuzzy C-Means.

1. K-Means Clustering

Definition

K-Means is an **unsupervised machine learning algorithm** used to group similar data points into clusters.

“K” means: Number of clusters.

Example

Suppose you have customer data:

- Age
- Salary

K-Means groups similar customers together like:

- Low salary customers
- Medium salary customers
- High salary customers

How K-Means Works

Step 1: Choose K (Choose number of clusters.)

Example:

$$K = 3$$

Means:

Create 3 clusters.

Step 2: Select Centroids

Algorithm randomly selects K center points.

Step 3: Calculate Distance

Each data point checks which centroid is nearest.

Usually uses:

Euclidean Distance:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Step 4: Form Clusters

Data points are grouped around nearest centroid.

Step 5: Update Centroids

New centroid is calculated for each cluster

Step 6: Repeat

Again calculate distance and regroup.

This repeats until centroids stop changing.

This is called Convergence

Data Points → Find nearest centroid → Form clusters

Advantages

- Simple
 - Fast
 - Easy to implement
-

Disadvantages

- Need to choose K value
- Sensitive to outliers
- Not good for irregular shapes

Why Elbow Method is Used?

- In K-Means, we must choose $n_clusters = ?$
- But the Problem is We don't know the correct K value.
- So we use **Elbow Method** to find the best K value.

We try different K values:

$K = 1$

$K = 2$

$K = 3$

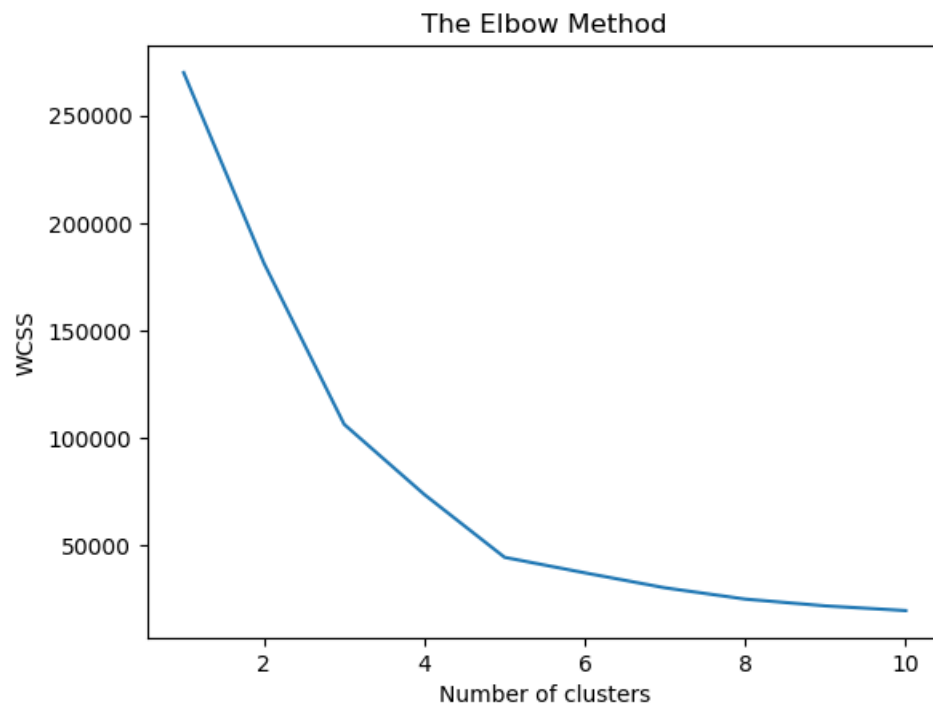
$K = 4 \dots$ Then calculate: WCSS (Within Cluster Sum of Squares)

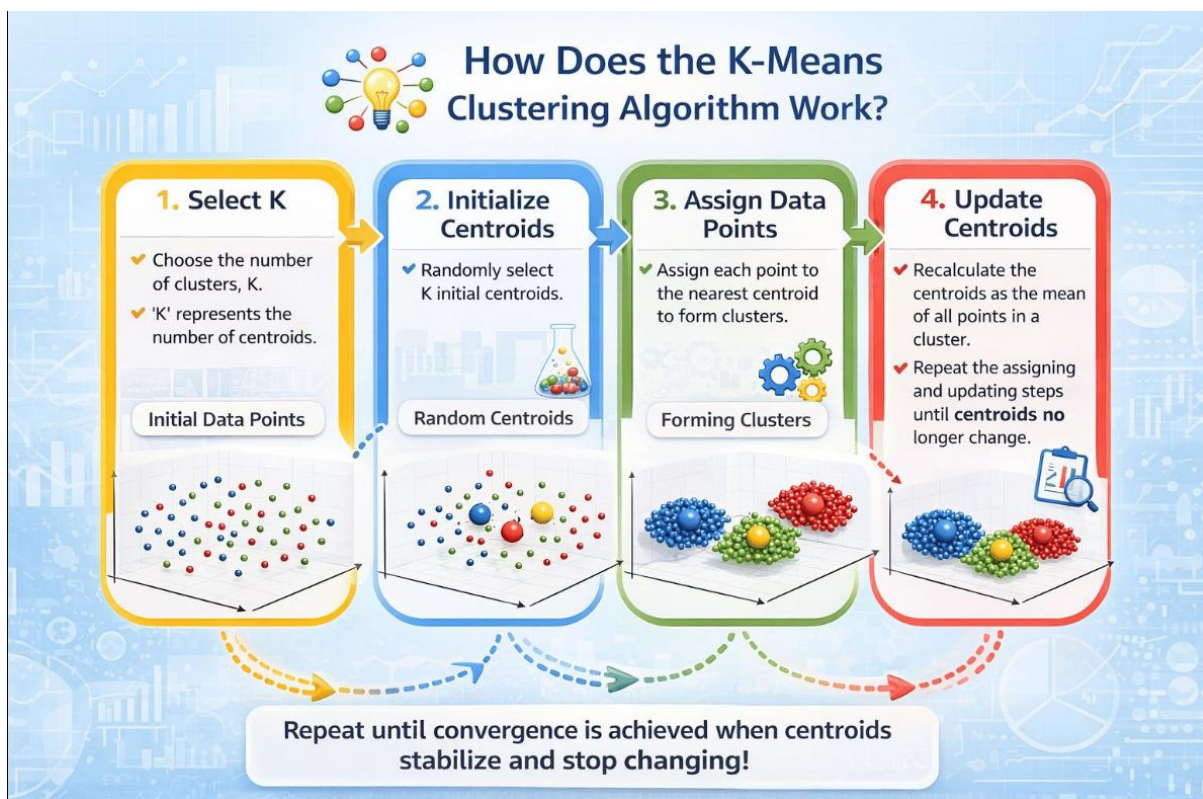
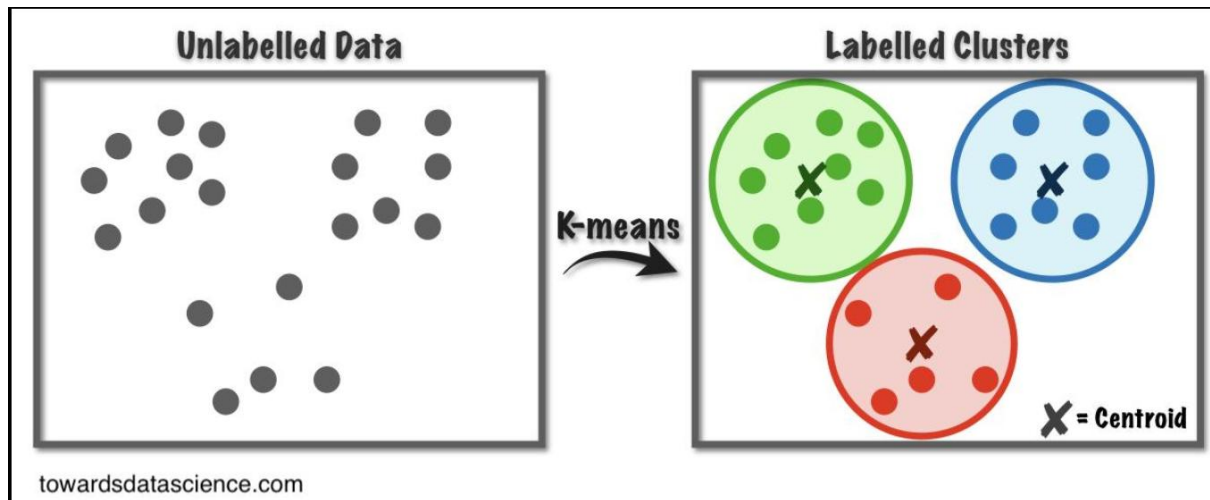
WCSS means: **Distance between data points and centroid.**

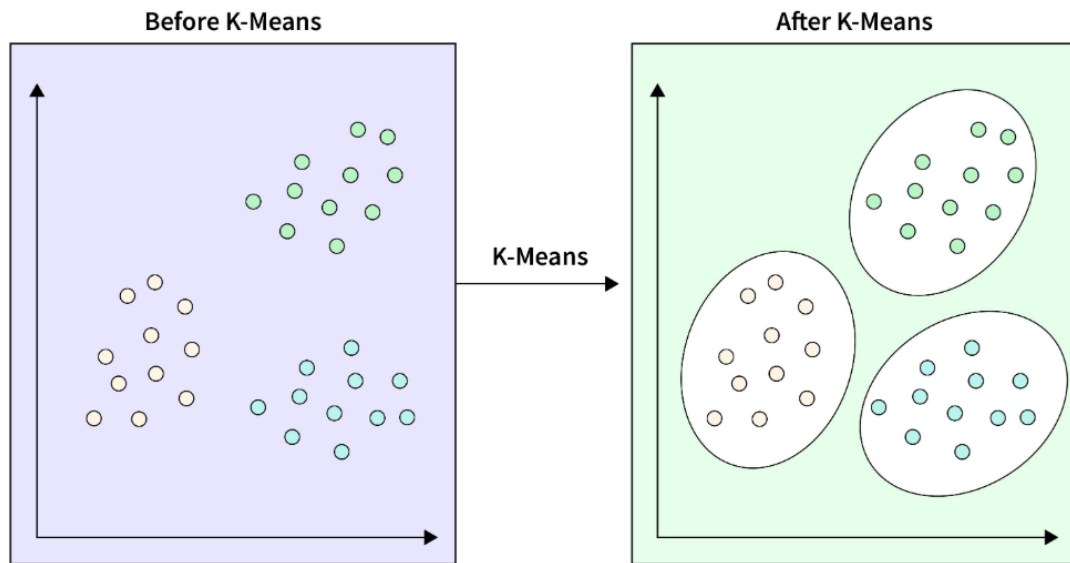
Smaller WCSS = better clustering.

SYNTAX:

```
from sklearn.cluster import KMeans
list1 = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', random_state = 42)
    kmeans.fit(X)
    list1.append(kmeans.inertia_)
plt.plot(range(1, 11), list1)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```







2. Agglomerative Clustering

Agglomerative Clustering is a type of Hierarchical Clustering

It follows Bottom-Up Approach

- Initially, every data point is treated as a separate cluster.
- Then nearest clusters are merged step by step.
- Process continues until all points become one cluster.

Example

Suppose we have 4 points:

A B C D

Initially:

[A] [B] [C] [D]

Step 1:

Nearest clusters merge

[AB] [C] [D]

Step 2:

[AB] [CD]

Step 3:

[ABCD]

How It Works:**Step 1**

Treat every point as individual cluster.

Step 2

Find nearest two clusters.

Step 3

Merge them.

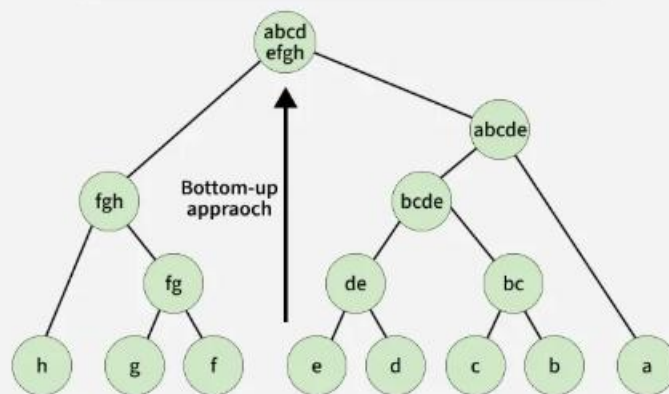
Step 4

Again find nearest clusters.

Step 5

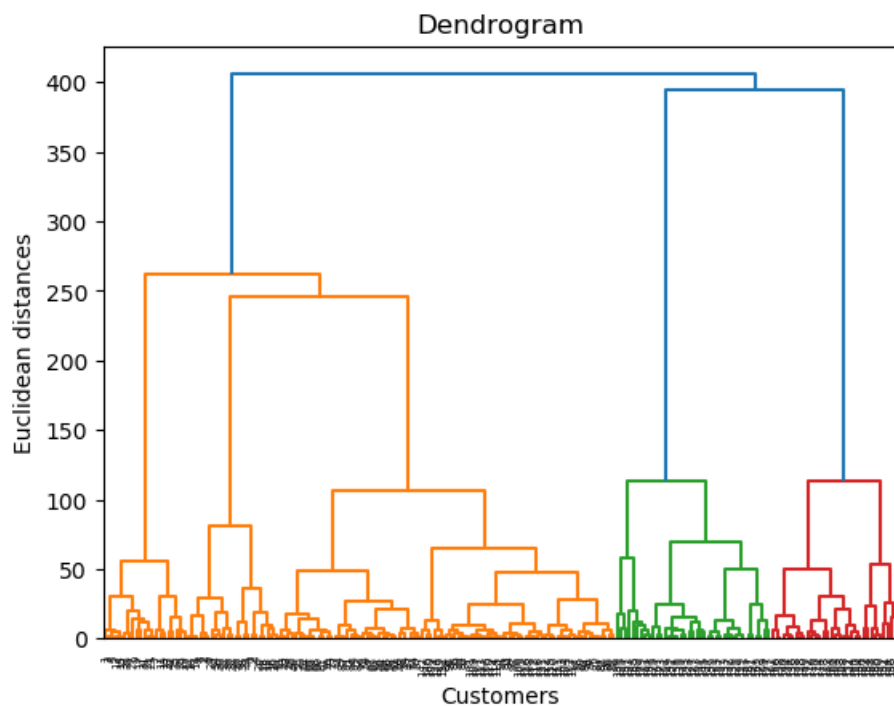
Repeat until one cluster remains.

Agglomerative Clustering

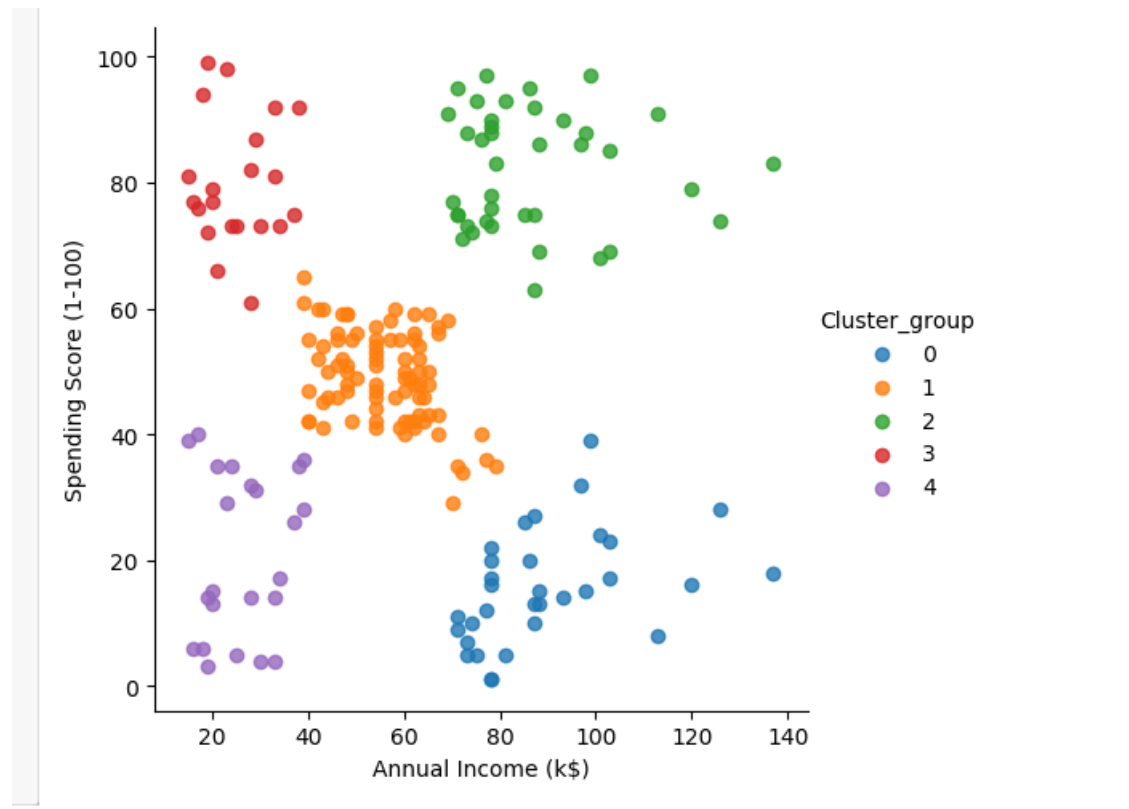


SYNTAX

```
In [8]: import scipy.cluster.hierarchy as sch
dendrogram = sch.dendrogram(sch.linkage(X,method='ward'))
plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean distances')
plt.show()
```



Output:



Advantages

- Easy to understand
- No random centroid issue
- Good visualization with dendrogram

Disadvantages

- Slow for large datasets
- More memory usage

Simply: *Agglomerative Clustering is a hierarchical clustering algorithm that starts with individual data points and repeatedly merges the nearest clusters until the required clusters are formed*

3.) Affinity Propagation Clustering

Affinity Propagation is a clustering algorithm that automatically finds clusters without specifying the number of clusters (K)

instead of using centroids like K-Means:

- Data points communicate with each other
- Important points are selected as “Exemplars”

Exemplar means:

Representative center point of a cluster.

Other points join the nearest exemplar.

Special Feature

- ✓ No need to give n_clusters

Algorithm automatically decides the number of clusters.

How It Works

It uses two types of messages:

1. Responsibility

Shows:

How suitable a point is to become exemplar.

2. Availability

Shows:

Whether other points support it becoming exemplar.

These messages are updated repeatedly until clusters are formed.

Model Flow

Data points communicate



Find exemplars



Form clusters

Advantages

- No need to choose K
- Automatically finds clusters
- Good for complex datasets

Disadvantages

- Slow for large datasets
- High memory usage

SYNTAX:

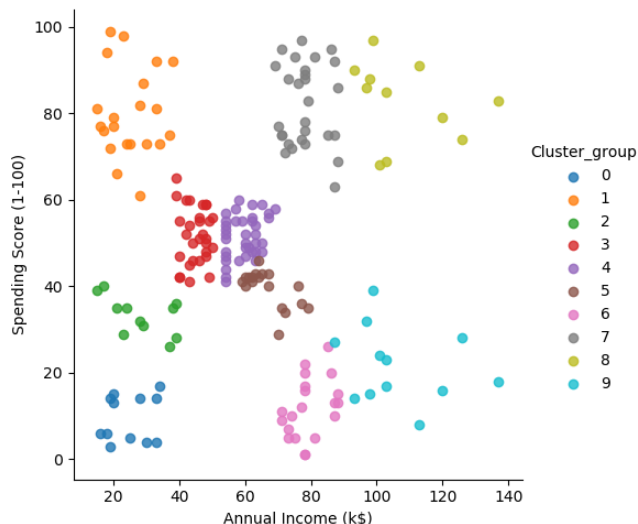
```
In [6]: from sklearn.cluster import AffinityPropagation
```

```
In [48]: af=AffinityPropagation(preference=-2500)
af.fit(X)
cluster_centers_indices = af.cluster_centers_indices_
labels = af.labels_
print(cluster_centers_indices)
print(labels)
```

OUTPUT:

```
In [52]: import seaborn as sns
facet = sns.lmplot(data=supervised, x=supervised.columns[3], y=supervised.columns[4], hue=supervised.columns[5],
                  fit_reg=False, legend=True, legend_out=True)
```

C:\Users\prave\anaconda3\envs\aiml\lib\site-packages\seaborn\regression.py:582: UserWarning: legend_out is deprecated from the `lmplot` function signature. Please update your code to pass it using `facet\_kws`.
warnings.warn(msg, UserWarning)



Simply ,

Affinity Propagation is a clustering algorithm that forms clusters by identifying representative data points called exemplars through message passing between data points.

4.)Mean Shift Clustering

Mean Shift is a clustering algorithm that groups data points by finding dense regions in the dataset.

It is a Density-Based Clustering algorithm.

- Algorithm finds areas where many data points are concentrated.
- It keeps shifting towards the region with highest density.
- Finally, clusters are formed around dense areas.

That is why it is called as a Mean Shift

Special Feature

- ✓ No need to specify “n_clusters”

Algorithm automatically finds number of clusters.

How It Works

Step 1

Place a window (circle) on data points.

Step 2

Calculate mean of points inside the window.

Step 3

Shift window toward the mean.

Step 4

Repeat shifting until center stops moving.

Step 5

Points converging to same area become one cluster.

Model Flow:

Find dense area



Move towards density



Form cluster

Important Parameter is “bandwidth” and it Controls window size.

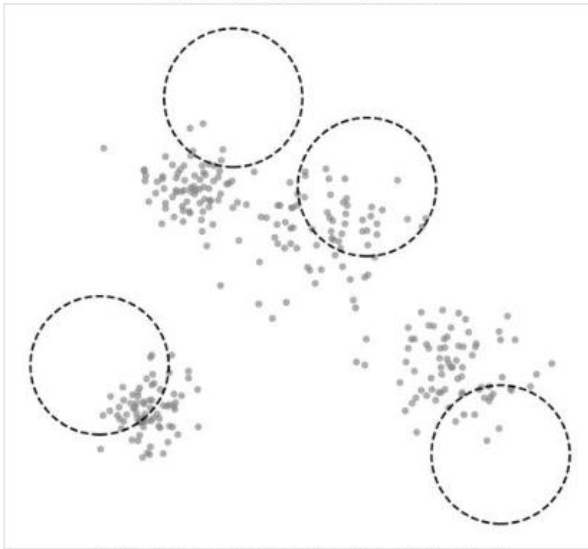
Example:

MeanShift(bandwidth=2)

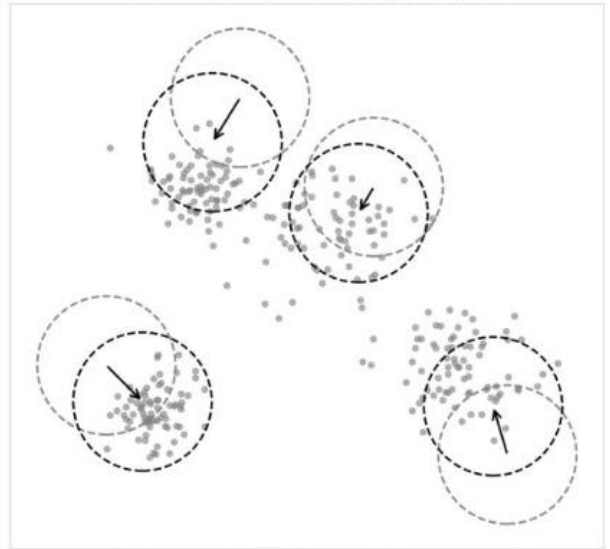
- Small bandwidth → more clusters
- Large bandwidth → fewer clusters

The Mean-Shift Algorithm: A Visual Walkthrough

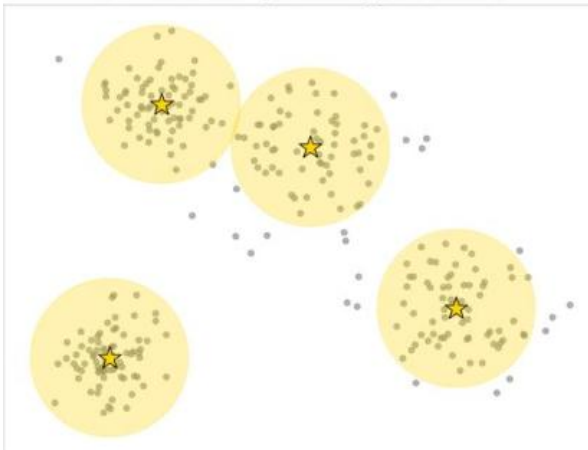
1. Start with initial search windows



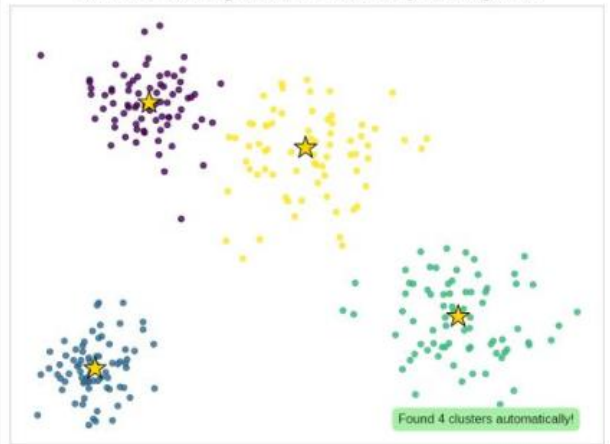
2. Shift windows to the mean of points within



3. Windows converge at density peaks (modes)

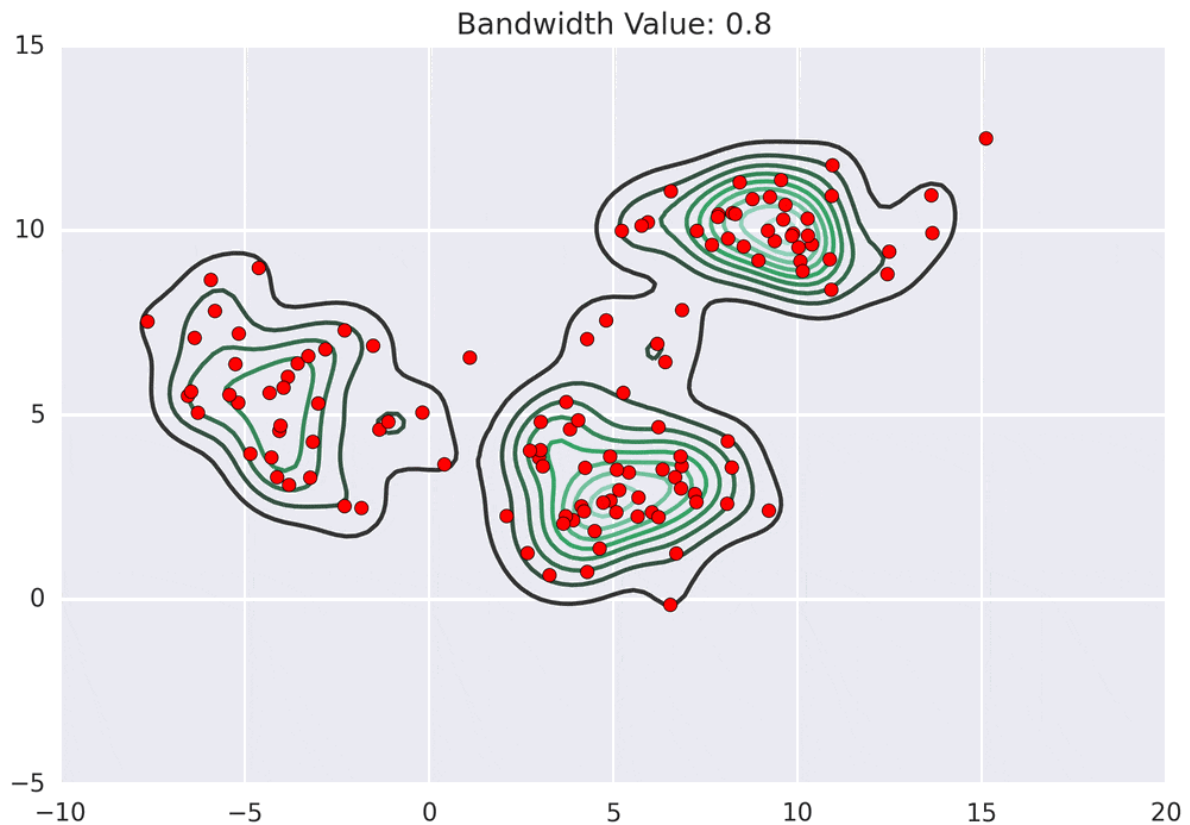


4. Points are assigned to the mode they converged to



Mean Shift Clustering Process





SYNTAX:

```
from sklearn.cluster import MeanShift
from sklearn.cluster import estimate_bandwidth
bandwidth = estimate_bandwidth(X, quantile=0.2)

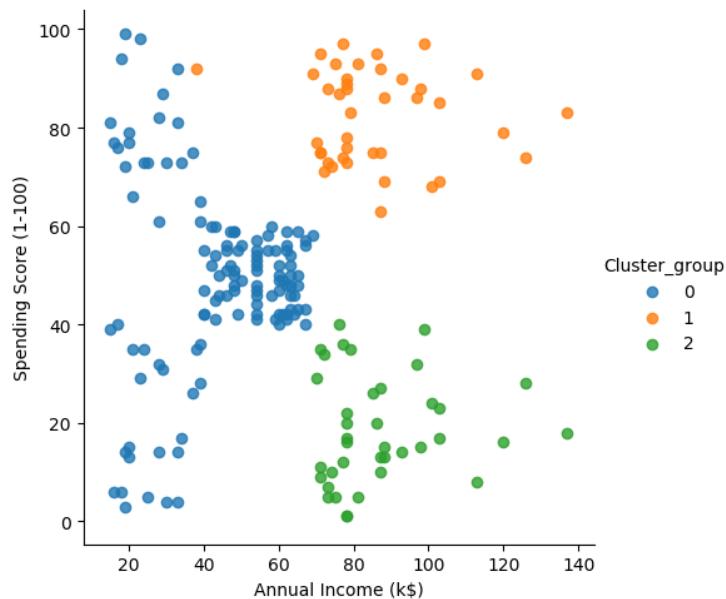
C:\Users\prave\anaconda3\envs\aiml\lib\site-packag
es are deprecated. Use packaging.version instead.
old_joblib = LooseVersion(joblib_version) < Loos
C:\Users\prave\anaconda3\envs\aiml\lib\site-packag
es are deprecated. Use packaging.version instead.
old_joblib = LooseVersion(joblib_version) < Loos

ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(X)
labels = ms.labels_
cluster_centers = ms.cluster_centers_
print(cluster_centers)
print(labels)
```

OUTPUT:

```
In [13]: import seaborn as sns
facet = sns.lmplot(data=supervised, x=supervised.columns[3], y=supervised.columns[4], hue=supervised.columns[5],
                  fit_reg=False, legend=True, legend_out=True)

C:\Users\prave\anaconda3\envs\aiml\lib\site-packages\seaborn\regression.py:582: UserWarning: legend_out is deprecated
`lmplot` function signature. Please update your code to pass it using `facet_kws`.
warnings.warn(msg, UserWarning)
```



Advantages

- No need to choose K
- Finds irregular shaped clusters
- Works well for dense regions

Disadvantages

- Slow for large datasets
- Sensitive to bandwidth value

Simply,

Mean Shift is a density-based clustering algorithm that identifies clusters by shifting data points toward regions of highest density.

5.)DBSCAN Clustering

DBSCAN stands for “Density-Based Spatial Clustering of Applications with Noise “It is a Density-Based Clustering algorithm.

DBSCAN groups:

- Closely packed data points → Cluster
- Far away points → Outliers / Noise

Special Feature

- ✓ Finds:
 - Irregular shaped clusters
 - Outliers automatically
- ✓ No need to specify “n_clusters”

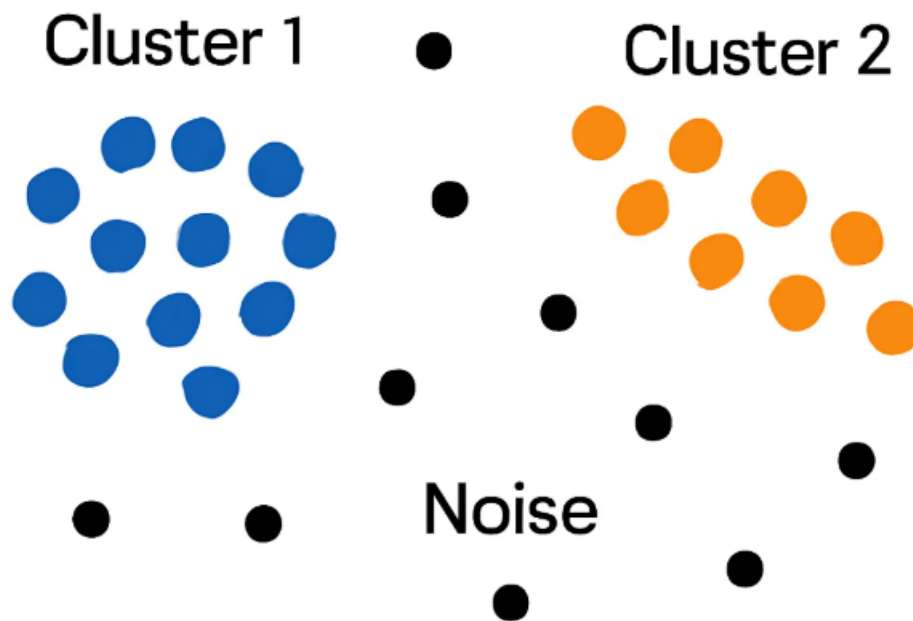
How DBSCAN Works

DBSCAN checks:

How many points are nearby.

If many points are close together → forms a cluster.

DBSCAN Clustering



Model Flow:

Check nearby points



Dense area → Cluster

Sparse area → Noise

SYNTAX:

```
In [12]: from sklearn import metrics
         from sklearn.cluster import DBSCAN

         db = DBSCAN(eps=10, min_samples=5)
         db.fit(X)
         labels = db.labels_
         n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
         n_noise_ = list(labels).count(-1)

         print("Estimated number of clusters: %d" % n_clusters_)
         print("Estimated number of noise points: %d" % n_noise_)
```

```
Estimated number of clusters: 4
Estimated number of noise points: 17
```

```
C:\Users\prave\anaconda3\envs\aiml\lib\site-packages\sklearn\neighbors\base.py:737: DeprecationWarning:
es are deprecated. Use packaging.version instead.
  if LooseVersion(joblib_version) < LooseVersion('0.12'):
C:\Users\prave\anaconda3\envs\aiml\lib\site-packages\sklearn\neighbors\base.py:737: DeprecationWarning:
es are deprecated. Use packaging.version instead.
  if LooseVersion(joblib_version) < LooseVersion('0.12'):
```

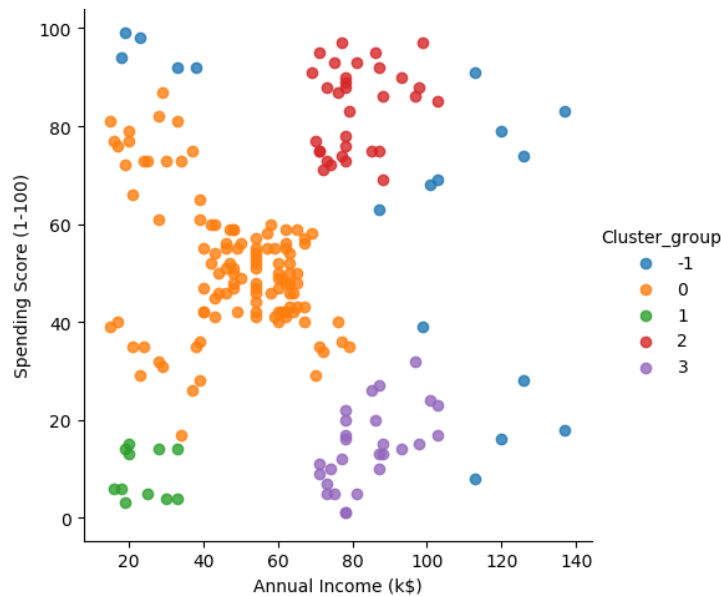
SYNTAX

```
from sklearn.cluster import DBSCAN
model = DBSCAN(eps=0.5,min_samples=5)
pred = model.fit_predict(X)
```

OUTPUT:

```
In [18]: import seaborn as sns
facet = sns.lmplot(data=supervised, x=supervised.columns[3], y=supervised.columns[4], hue=supervised.columns[5],
                  fit_reg=False, legend=True, legend_out=True)
plt.show()

C:\Users\prave\anaconda3\envs\aiml\lib\site-packages\seaborn\regression.py:582: UserWarning: legend_out is deprecated in
`lmplot` function signature. Please update your code to pass it using `facet_kws`.
warnings.warn(msg, UserWarning)
```



1. Epsilon (eps)

Radius distance.

Inside this radius, nearby points are checked.

2. Min Samples (min_samples)

Minimum number of nearby points needed to form cluster.

Example:

`min_samples = 5`

Means ,At least 5 nearby points required.

Advantages

- Finds outliers automatically
- Handles irregular shapes
- No need K value

Disadvantages

- Difficult to choose eps
- Struggles with varying densities

Simply,

DBSCAN is a density-based clustering algorithm that forms clusters from densely packed data points and identifies sparse points as outliers.

Clustering Algorithms Overview:

Algorithm	Type	Main Idea	Need K Value	Important Parameters	Special Feature  Share
K-Means	Partition-Based	Groups data using centroids	✔ Yes	<code>n_clusters</code>	Fast and simple
Affinity Propagation	Message-Based	Finds exemplars using message passing	✗ No	<code>damping</code>	Automatically finds clusters
Mean Shift	Density-Based	Moves toward dense regions	✗ No	<code>bandwidth</code>	Finds dense areas automatically
Spectral Clustering	Graph-Based	Uses graph similarity and eigenvalues	✔ Yes	<code>n_clusters</code>	Good for complex shapes
Hierarchical Clustering	Hierarchical	Merges clusters step by step	✔ Usually Yes	<code>n_clusters</code> , <code>linkage</code>	Uses dendrogram
DBSCAN	Density-Based	Groups dense points and finds noise	✗ No	<code>eps</code> , <code>min_samples</code>	Detects outliers
HDBSCAN	Density-Based	Improved DBSCAN using varying density	✗ No	<code>min_cluster_size</code>	Better for different densities
OPTICS	Density-Based	Orders points based on density	✗ No	<code>min_samples</code> , <code>max_eps</code>	Handles varying densities
BIRCH	Hierarchical / Tree-Based	Builds clustering feature tree	✔ Optional	<code>threshold</code> , <code>n_clusters</code>	Very fast for large datasets
Clustering Performance Evaluation	Evaluation Method	Measures clustering quality	—	Silhouette Score, Davies-Bouldin Score	Evaluates cluster performance

K-Means

Uses centroids to divide data into K clusters.

Affinity Propagation

Finds representative points called exemplars automatically.

Mean Shift

Finds clusters by shifting toward dense regions.

Spectral Clustering

Uses graph similarity matrix for clustering complex-shaped data.

Hierarchical Clustering

Creates clusters in tree structure using merging.

DBSCAN

Groups dense points and detects outliers/noise.

HDBSCAN

Advanced version of DBSCAN that handles varying densities better.

OPTICS

Similar to DBSCAN but works better when densities vary.

BIRCH

Efficient clustering for very large datasets using tree structure.

Most Commonly Used Algorithms

- K-Means
- Hierarchical
- DBSCAN
- Mean Shift

Advanced Algorithms

- Spectral Clustering
- HDBSCAN
- OPTICS
- BIRCH